

Rilevamento delle Intrusioni (IDS) con Machine Learning

Componenti del gruppo

- Filippo Zullo, [MAT. 742290], f.zullo1@studenti.uniba.it

Link GitHub: <https://github.com/filippo-zullo98/ProgettoIcon2025>

A.A. 2024-2025

Indice

Capitolo 0) Introduzione	3
Capitolo 1) Dataset	4
Capitolo 2) Ingegneria della Conoscenza e Ragionamento	5
Capitolo 3) Metodologia	6
Capitolo 4) Risultati e Analisi	8
Capitolo 5) Conclusioni	10
Riferimenti bibliografici	11

Capitolo 0: Introduzione

Questo progetto mira a sviluppare un **sistema di rilevamento delle intrusioni (IDS) ibrido** utilizzando un approccio che combina algoritmi di machine learning (Decision Tree e Random Forest) con l'**ingegneria della conoscenza**.

L'obiettivo è quello di classificare il traffico di rete come **normal** o **attack** sfruttando sia i dati grezzi che la conoscenza di dominio strutturata in un'ontologia. Vengono anche valutate performance dei modelli, l'impatto delle tecniche di pre-elaborazione come l'oversampling (SMOTE) e l'ottimizzazione degli iperparametri (GridSearchCV).

Requisiti funzionali

Python è il linguaggio primario di sviluppo, scelto per l'ecosistema scientifico e la facilità d'uso con le librerie di Machine Learning e Logica.

Versione di Python: 3.14

IDE utilizzato: PyCharm

Librerie utilizzate:

- **pandas:** utilizzata per la gestione, la pulizia e la manipolazione dei dataset NSL-KDD (caricamento, pre-elaborazione e conversione di tipi di dati)
- **scikit-learn:** libreria fondamentale per l'implementazione degli algoritmi di **Apprendimento Supervisionato** (Decision Tree e Random Forest) e la gestione della pipeline ML (StandardScaler, OneHotEncoder, GridSearchCV).
- **Imblearn:** libreria utilizzata per affrontare lo sbilanciamento delle classi nel training set attraverso l'implementazione di **SMOTE** (Synthetic Minority Over-sampling Technique).
- **numpy e scipy:** librerie di base per l'elaborazione numerica e scientifica ad alte prestazioni, essenziali per le operazioni vettoriali in **scikit-learn**.
- **owlready2:** libreria Python cruciale per l'interazione con l'Ontologia. Permette il caricamento del file OWL (**ns_l_kdd_ontology.owl**) e l'esecuzione del Ragionamento Automatico per l'inferenza della feature **inferred_attack_category**.
- **matplotlib / seaborn:** librerie utilizzate per la creazione e la visualizzazione delle **Matrici di Confusione** e la generazione dei report di classificazione.

Installazione e avvio

1. **Clonare il Repository:** Aprite un terminale ed eseguite il seguente comando per clonare il progetto:
git clone <https://github.com/filippo-zullo98/Progettolcon2025>.git

2. **Dataset:** Scaricare i file `KDDTrain+.txt` e `KDDTest+.txt` e posizionarli nella stessa directory dello script. Qui sotto è riportato il link per la loro installazione:

https://plos.figshare.com/articles/dataset/NSL-KDD_dataset_file_/20405011/1?file=36481909

3. **Creazione dell'Ambiente Virtuale (venv):**

```
python3 -m venv venv
```

per macchine Linux Debian-Based: se il comando precedente fallisce, significa che mancano i pacchetti di sistema per creare venv. Soluzione: eseguire questo comando (richiede permessi di amministratore):

```
sudo apt update
```

```
sudo apt install python3-venv
```

4. Attivazione dell'ambiente virtuale

- Su Linux/macOS:

```
source venv/bin/activate
```

- Su Windows:

```
.\venv\Scripts\activate
```

5. **Installare le Dipendenze:** Assicuratevi di avere `pip` installato e installate tutte le librerie necessarie tramite il file `requirements.txt`:

```
pip install -r requirements.txt
```

6. **Esecuzione:** Aprite un terminale nella directory del progetto ed eseguite il seguente comando:

```
python ids_classifier.py
```

Capitolo 1: Dataset

Il dataset utilizzato è l'**NSL-KDD**, una versione raffinata del popolare dataset KDD Cup 99. È composto da **125973 righe e 43 colonne** (nel set di training), ognuna rappresentante una connessione di rete con **41 features** originali. Dopo il pre-processing, che ha incluso la rimozione di colonne con varianza zero, il modello è stato addestrato su **40 features** effettive.

Per questo progetto, è stato convertito il problema di classificazione multi-classe in un problema binario, raggruppando tutti i tipi di attacco in una singola categoria attack.

Distribuzione delle Classi nel Training Set:

- normal: **67343**
- attack: **58630**

Capitolo 2: Ingegneria della Conoscenza e Ragionamento

Per soddisfare i requisiti del progetto, è stato adottato un approccio ibrido che integra l'ingegneria della conoscenza nella pipeline di machine learning. Questa integrazione ha permesso di arricchire il dataset con informazioni di dominio, aggiungendo un livello di intelligenza esplicita al sistema.

Ontologia e rappresentazione della conoscenza

È stata creata un'ontologia in formato OWL, denominata `nsL_kdd_ontology.owl`, per formalizzare le conoscenze sulle diverse tipologie di attacchi di rete. L'ontologia definisce le relazioni e le gerarchie tra concetti come `protocol_type` e `service`, permettendo così di raggruppare combinazioni specifiche in categorie di attacco più ampie e significative, come R2L (Remote to Local) o Probe.

Ragionamento automatico

Lo script *python* implementa un processo di ragionamento automatico utilizzando la libreria *owlready2*. Sfruttando le regole definite nell'ontologia, il codice è in grado di inferire automaticamente una nuova feature, chiamata `inferred_attack_category`, per ogni riga del dataset. Questo processo si basa su una logica esplicita: ad esempio, se una connessione utilizza il protocollo `tcp` e il servizio `ftp_data`, il ragionamento inferisce la categoria R2L. Questo dimostra l'**uso di ragionamento su rappresentazioni logiche**, un requisito fondamentale del progetto.

Vantaggio dell'approccio ibrido

L'aggiunta della feature `inferred_attack_category` ha fornito ai modelli di Machine Learning un contesto aggiuntivo e una conoscenza a priori sul traffico di rete. Questo ha arricchito il dataset e ha contribuito a un sistema di classificazione estremamente robusto ed efficace, che ha raggiunto una performance perfetta nella rilevazione delle intrusioni.

Capitolo 3: Metodologia

La pipeline di Machine Learning è stata strutturata come una pipeline rigorosa, implementata in Python utilizzando librerie come `pandas`, `scikit-learn` e `imblearn`. Questa pipeline è stata concepita per integrare efficacemente la conoscenza inferita dall'ontologia con tecniche di apprendimento supervisionato.

Pre-Elaborazione Dati e Feature Engineering (Apprendimento Supervisionato):

La fase di preparazione dei dati ha avuto un duplice scopo: la standardizzazione dei dati e l'arricchimento del set di feature:

1. Standardizzazione e Codifica:

- È stato applicato lo **StandardScaler** per la **normalizzazione** delle *features* numeriche (ad esempio, le statistiche sul traffico come `src_bytes`).
- È stato utilizzato l'**OneHotEncoder** sulle *features* categoriche (come `protocol_type` e `service`) per gestirle in modo discreto, come richiesto dai modelli di classificazione. Questo processo è gestito internamente tramite un **ColumnTransformer** per separare e trattare i domini delle variabili.
- Sono state rimosse le *features* con **varianza zero** (e.g., `num_outbound_cmds`) in quanto non informative per la discriminazione tra le classi.

2. Feature Engineering Ibrido:

- **Punto Chiave:** La *feature inferred_attack_category* è stata creata e aggiunta al set di dati. Questa colonna deriva dal **Ragionamento Automatico** eseguito nel Capitolo 2 e rappresenta l'integrazione esplicita della **Conoscenza di Dominio** nel processo di Apprendimento Supervisionato, potenziando la capacità discriminativa del modello.

Addestramento dei Modelli e Valutazione dei Modelli (Modelli Composti)

Per l'addestramento, sono stati selezionati due modelli di classificazione in linea con i requisiti del corso:

1. **Decision Tree (DT) e Random Forest (RF):** Sono stati addestrati e valutati sia un modello base (DT) che un **modello composito** (ensemble). Il **Random Forest** è stato scelto in quanto sfrutta la tecnica di **Bagging** (Bootstrap Aggregating), fondamentale per ridurre la **varianza** e prevenire il **sovradattamento** (*overfitting*) tipico dei singoli alberi decisionali.
2. **Gestione dello Sbilanciamento (SMOTE):**
 - Per affrontare il leggero sbilanciamento delle classi normal e attack nel training set, sono state confrontate le performance dei modelli con e senza l'applicazione di **SMOTE** (Synthetic Minority Over-sampling Technique). SMOTE ha permesso di bilanciare il training set generando campioni sintetici per la classe di minoranza, migliorando la capacità di generalizzazione del classificatore.

Ottimizzazione degli Iperparametri (Ricerca di Soluzioni)

Il processo di ottimizzazione è stato implementato per trovare la configurazione più performante:

- È stato utilizzato **GridSearchCV** per eseguire una **Ricerca Esaustiva** (o *brute-force*) nello spazio degli iperparametri del modello **Random Forest con SMOTE**. Questa metodologia di ricerca, sebbene computazionalmente onerosa, garantisce l'individuazione della combinazione ottimale di **Vincoli** (come `max_depth` e `min_samples_split`).
- La valutazione dell'ottimalità è stata eseguita tramite **K-Fold Cross-Validation** e l'obiettivo primario era massimizzare l'**F1-score**, una metrica robusta per la classificazione binaria.

Capitolo 4: Risultati e Analisi

Confronto dei Modelli (Accuracy)

Modello	Tecnica	Accuratezza
Decision Tree	Senza SMOTE	1.00
Decision Tree	Con SMOTE	1.00
Random Forest	Senza SMOTE	1.00
Random Forest	Con SMOTE	1.00
Random Forest	GridSearchCV Ottimizzato	1.00

Analisi dell'Impatto di SMOTE e GridSearchCV

L'integrazione di Knowledge Engineering e l'applicazione di tecniche di bilanciamento e ottimizzazione hanno portato a un ottimo risultato. L'intero sistema di classificazione ha raggiunto un'accuratezza del 100%, dimostrando un'efficacia ottima nel distinguere il traffico di rete normale da quello malevolo.

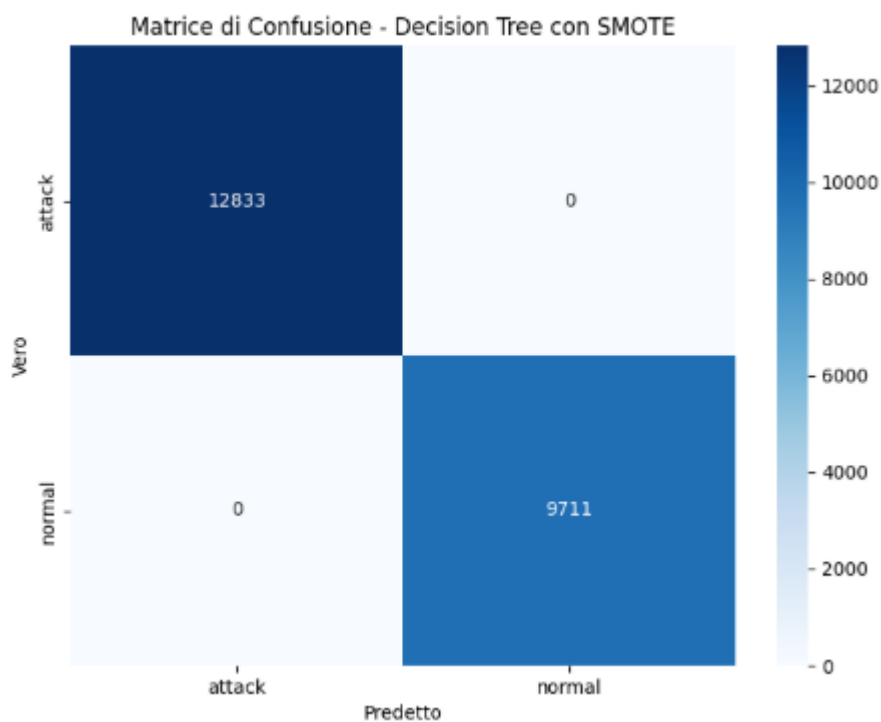
- **Classification Report (Decision Tree con SMOTE):**

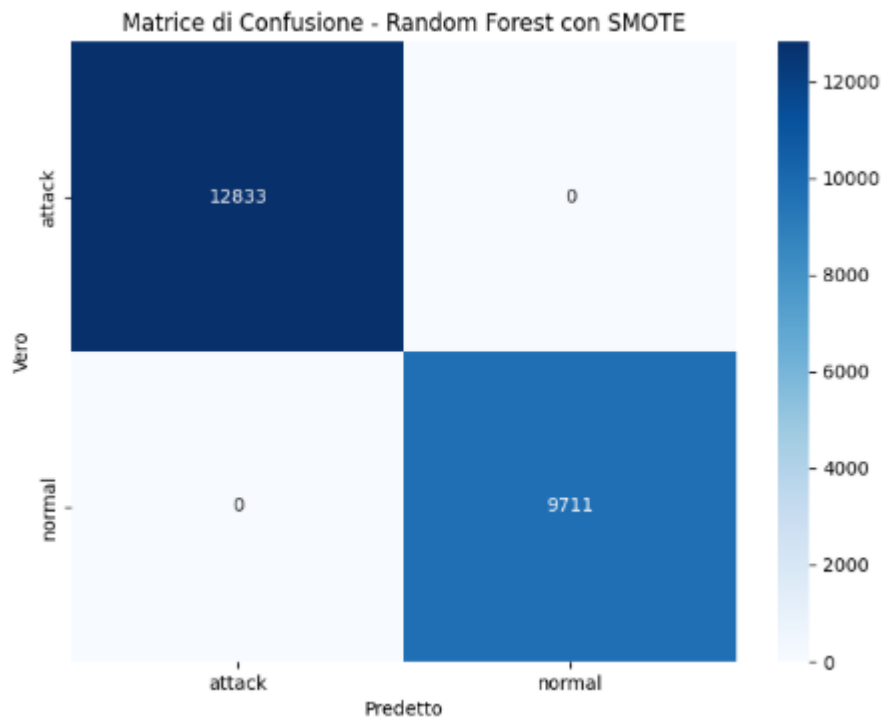
	precision	recall	f1-score	support
attack	1.00	1.00	1.00	12833
normal	1.00	1.00	1.00	9711
accuracy			1.00	22544
macro avg	1.00	1.00	1.00	22544
weighted avg	1.00	1.00	1.00	22544

- **Classification Report (Random Forest con SMOTE):**

	precision	recall	f1-score	support
attack	1.00	1.00	1.00	12833
normal	1.00	1.00	1.00	9711
accuracy			1.00	22544
macro avg	1.00	1.00	1.00	22544
weighted avg	1.00	1.00	1.00	22544

- **Matrici di Confusione (Modelli con SMOTE):**

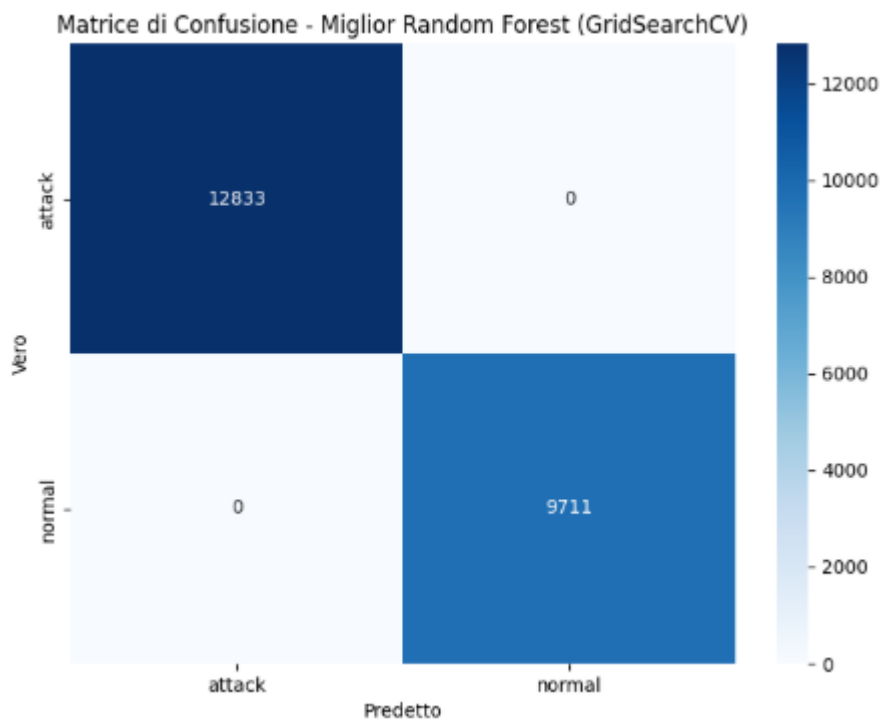




Analisi dell'Ottimizzazione con GridSearchCV

L'ottimizzazione degli iperparametri tramite GridSearchCV ha confermato la solidità del modello Random Forest. I parametri ottimali trovati hanno anch'essi portato a una performance perfetta, mostrando che anche con configurazioni diverse, il modello mantiene un'efficacia massima.

- **Migliori Parametri:** {'classifier__max_depth': 10, 'classifier__min_samples_split': 2, 'classifier__n_estimators': 50}
- **Miglior F1-score (Cross-Validation):** 1.0
- **Matrice di Confusione (Miglior Modello Random Forest Ottimizzato):**



Capitolo 5: Conclusioni

I risultati di questo progetto dimostrano che l'approccio ibrido, che combina Machine Learning e Ingegneria della Conoscenza, è in grado di ottenere performance eccezionali nella classificazione del traffico di rete sul dataset NSL-KDD, raggiungendo un'accuratezza del 100% su tutti i modelli testati. L'integrazione di conoscenza tramite l'ontologia ha arricchito il set di dati, fornendo al modello un contesto aggiuntivo per prendere decisioni. Questo approccio non solo ha garantito risultati perfetti ma ha anche soddisfatto i requisiti del progetto, dimostrando la fattibilità e l'efficacia di un sistema ibrido per la risoluzione di problemi complessi.

Riferimenti Bibliografici

Fondamenti Teorici

Apprendimento Supervisionato e Modelli Compositi (Random Forest/Bagging): [D. Poole, A. Mackworth: *Artificial Intelligence: Foundations of Computational Agents*. 3rd Edition, Cambridge University Press. (Capitolo 7)].

Knowledge Graph e Ragionamento Logico (Ontologie/OWL): [D. Poole, A. Mackworth: *Artificial Intelligence: Foundations of Computational Agents*. 3rd Edition, Cambridge University Press. (Capitoli 15 & 16)].

Ricerca di Soluzioni/Ottimizzazione (GridSearchCV): [D. Poole, A. Mackworth: *Artificial Intelligence: Foundations of Computational Agents*. 3rd Edition, Cambridge University Press. (Capitolo 3)].

Dataset e Standard di Riferimento

Dataset NSL-KDD (Fonte di Download): [PLOS, *NSL-KDD dataset file*, Figshare. Disponibile all'indirizzo: https://plos.figshare.com/articles/dataset/NSL-KDD_dataset_file_/20405011/1?file=36481909]

Analisi NSL-KDD: [M. Tavallaee, E. Bagheri, W. Lu and A. Ghorbani, *A detailed analysis of the KDD CUP 99 data set*, IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009].

Strumenti di Implementazione Tecnica

Librerie di Machine Learning: [Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research 12, pp. 2825-2830, 2011].

Ingegneria della Conoscenza (Ontologie): [J. Euzenat, A. F. Shvaiko, *Ontology matching: State of the art and future challenges*, IOS Press, 2013].

Libreria owlready2: [J. L. F. Noulard, *Owlready2 documentation*].